

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования

**«Пермский национальный исследовательский политехнический
университет»**

Электротехнический факультет

Выпускающая кафедра: Информационные технологии и
автоматизированные системы (ИТАС)

Направление подготовки: 09.03.04 Программная инженерия

ОТЧЕТ

Лабораторная работа №1

«Классы и объекты. Инкапсуляция.»

По дисциплине «Основы алгоритмизации и программирования»

Вариант 15

Выполнил: студент группы РИС-25-2Б

Ильинов Фёдор Михайлович

Принял: Доц. Полякова О. А.

Пермь 2026

ПОСТАНОВКА ЗАДАЧИ

Реализовать определение нового класса. Для демонстрации работы с объектами написать главную функцию. Продемонстрировать разные способы создания объектов и массивов объектов.

2. Структура-пара – структура с двумя полями, которые обычно имеют имена `first` и `second`. Требуется реализовать тип данных с помощью такой структуры. Во всех заданиях должны присутствовать :

а. метод инициализации `Init` (метод должен контролировать значения аргументов на корректность);

б. ввод с клавиатуры `Read`;

с. вывод на экран `Show`.

3. Реализовать внешнюю функцию `make_тип()`, где `тип` – тип реализуемой структуры. Функция должна получать значения для полей структуры как параметры функции и возвращать структуру как результат. При передаче ошибочных параметров следует выводить сообщение и заканчивать работу.

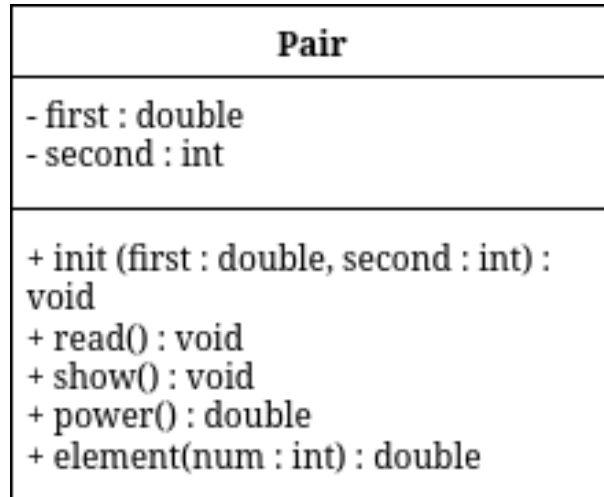
АНАЛИЗ.

1. Согласно принципам инкапсуляции поля класса должны быть приватными, а описанные в постановке методы — публичными.

2. `Pair makePair` должна принимать параметры `first`, `second` и затем передавать их в метод `Pair::init()`

3. Ввод/вывод реализуем при помощи `std::cin`, `std::cout`

UML-ДИАГРАММА



КОД Pair.h

```
#ifndef ILINOV_FEDOR_RIS_25_2B_LABS_PSTU_CS_FRACTION_H
#define ILINOV_FEDOR_RIS_25_2B_LABS_PSTU_CS_FRACTION_H

class Pair {
    double first;
    int second;
public:
    void init(double, int);
    void read();
    void show();
    double power();
    double element(int);
};

#endif //ILINOV_FEDOR_RIS_25_2B_LABS_PSTU_CS_FRACTION_H
```

КОД Pair.cpp

```
#include "Pair.h"
#include <iostream>

void Pair::init(double a, int b) {
    first = a;
    second = b;
}
```

```

}

void Pair::read() {
    std::cout << "First? ";
    std::cin >> first;
    std::cout << "Second? ";
    std::cin >> second;

}

void Pair::show() {
    std::cout << "First: " << first << "\nSecond: " << second << '\n';
}

double Pair::power() {
    double ans = first;
    for (int i = 1; i < second; i++) ans *= first;
    return ans;
}

double Pair::element(int a) {
    Pair tmp;
    tmp.init(second, a);
    return first * tmp.power();
}

```

Код main.cpp

```

#include <iostream>
#include "Pair.h"

using namespace std;

Pair makePair(int i, double d) {
    Pair p;
    p.init(d, i);
    return p;
}

int main() {
    Pair f;
    f.read();
}

```

```
f.show();  
cout << "a^b = " << f.power() << "\n";  
cout << "a[3] = " << f.element(3) << "\n";  
cout << "a.init(3, 2);\n";  
f.init(3, 2);  
f.show();  
}
```

РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ

First? 3.14

Second? 2

First: 3.14

Second: 2

$a^b = 9.8596$

$a[3] = 25.12$

a.init(3, 2);

First: 3

Second: 2

ВОПРОСЫ

1. Что такое класс?

Класс - абстрактный пользовательский тип данных, представляющий собой модель реального объекта в виде данных (полей) и функций (методов) для работы с ними. Вместе поля и методы называются элементами класса.

2. Что такое объект (экземпляр) класса?

Экземпляр класса - переменная (константа) в коде, имеющая тип класса.

3. Как называются поля класса?

Атрибуты

4. Как называются функции класса?

Методы

5. Для чего используются спецификаторы доступа?

Для сокрытия внутренних атрибутов и методов от пользователя.

6. Для чего используется спецификатор public?

Элементы класса, описанные в поле "public" доступны для использования вне класса при помощи операторов
subject.field (или subject->field, если subject - указатель)

7. Для чего используется спецификатор private?

Элементы класса, описанные в поле "private" не доступны для использования вне класса при помощи операторов
subject.field (или subject->field, если subject - указатель)

8. Если описание класса начинается со спецификатора class, то какой спецификатор доступа будет использоваться по умолчанию?

private

9. Если описание класса начинается со спецификатора struct, то какой спецификатор доступа будет использоваться по умолчанию?

public

10. Какой спецификатор доступа должен использоваться при описании интерфейса класса? Почему?

В public, так как интерфейс подразумевает использование самого себя во вне класса.

11. Каким образом можно изменить значения атрибутов экземпляра класса?

При помощи операторов subject.field (или subject->field, если subject - указатель)

12. Каким образом можно получить значения атрибутов экземпляра класса?
При помощи операторов `subject.field` (или `subject->field`, если `subject` - указатель)

13. Класс описан следующим образом

```
struct Student
{
    string name;
    int group;
    .....
};
```

Объект класса определен следующим образом

```
Student *s=new Student;
```

Как можно обратиться к полю `name` объекта `s`?

```
s->name;
```

14. Класс описан следующим образом

```
struct Student
{
    string name;
    int group;
    .....
};
```

Объект класса определен следующим образом

```
Student s;
```

Как можно обратиться к полю `name` объекта `s`?

```
s.name
```

15. Класс описан следующим образом

```
class Student
{
    string name;
    int group;
    .....
};
```

Объект класса определен следующим образом

```
Student *s=new Student;
```

Как можно обратиться к полю `name` объекта `s`?

Никаким. `name` - приватное поле

16. Класс описан следующим образом

```
class Student
{
    string name;
    int group;
    public:
```

```
.....  
};  
Объект класса определен следующим образом  
Student s;  
Как можно обратиться к полю name объекта s?  
Никаким. name - приватное поле
```

17. Класс описан следующим образом

```
class Student
```

```
{  
    public:  
    char* name;  
    int group;  
    .....
```

```
};  
Объект класса определен следующим образом  
Student *s=new Student;  
Как можно обратиться к полю name объекта s?  
Например cin >> s->name;
```